# Web 3.0 Unveiled - Day 3

Mentored by FEC

## Introduction to Solidity


- Solidity is an object-oriented, high-level language for implementing smart contracts. It is designed to target Ethereum Virtual Machine(EVM)
- It is statically typed, supports inheritance, libraries and complex user-defined types among other features.

## Building in Solidity

### For those who prefer videos:

- Learn Solidity

### For those who prefer reading:

#### Initializing smart contracts

```
// Define the compiler version you would be using
pragma solidity ^0.8.10;

// Start by creating a contract named HelloWorld
```

```
contract HelloWorld {

}
```

## Variables and types

There are 3 types of variables in Solidity

- Local
    - Declared inside a function and are not stored on the blockchain
- State
    - Declared outside a function to maintain the state of the smart contract
    - Stored on the blockchain
- Global
    - Provide information about the blockchain. They are injected by the Ethereum Virtual Machine during runtime.
    - Includes things like transaction sender, block timestamp, block hash, etc.
    - Examples of global variables

The scope of variables is defined by where they are declared, not their value. Setting a local variable's value to a global variable does not make it a global variable, as it is still only accessible within its scope.

```
// Define the compiler version you would be using
pragma solidity ^0.8.10;


// Start by creating a contract named Variables
contract Variables {
   /*
       ******** State variables **********
   */
   /*
   uint stands for unsigned integer, meaning non-negative integers
   different sizes are available. Eg
       - uint8   ranges from 0 to 2 ** 8 - 1
       - uint256 ranges from 0 to 2 ** 256 - 1
   `public` means that the variable can be accessed internally
    by the contract and can also be read by the external world
   */
   uint8 public u8 = 10;
   uint public u256 = 600;
   uint public u = 1230; // uint is an alias for uint256
```

```
    /*
    Negative numbers are allowed for int types. Eg
    - int256 ranges from -2 ** 255 to 2 ** 255 - 1
    */
    int public i = -123; // int is same as int256


    // address stands for an Ethereum address
    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;


    // bool stands for boolean
    bool public defaultBoo1 = false;


    // Default values
    // Unassigned variables have a default value in Solidity
    bool public defaultBoo2; // false
    uint public defaultUint; // 0
    int public defaultInt; // 0
    address public defaultAddr; // 0x0000000000000000000000000000000000000000


    function doSomething() public {
        /*
        ******** Local variable **********
        */
        uint ui = 456;


        /*
        ******** Global variables **********
        */


        /*
            block.timestamp tells us whats the timestamp for the current block
            msg.sender tells us which address called the doSomething function
        */
        uint timestamp = block.timestamp; // Current block timestamp
        address sender = msg.sender; // address of the caller
    }
}
```

## Functions, Loops, and If/Else

```
// Define the compiler version you would be using
pragma solidity ^0.8.10;


// Start by creating a contract named Conditions
contract Conditions {
    // State variable to store a number
    uint public num;


    /*
        Name of the function is set.
        It takes in a uint and sets the state variable num.
        It is declared as a public function meaning
        it can be called from within the contract and also externally.
    */
    function set(uint _num) public {
        num = _num;
    }


    /*
        Name of the function is get.
        It returns the value of num.
        It is declared as a view function meaning
        that the function doesn't change the state of any variable.
        View functions in solidity do not require gas.
    */
    function get() public view returns (uint) {
        return num;
    }


    /*
        Name of the function is foo.
        It takes in a uint and returns a uint.
        It compares the value of x using if/else
    */
    function foo(uint x) public returns (uint) {
        if (x < 10) {
            return 0;
```

```
        } else if (x < 20) {
            return 1;
        } else {
            return 2;
        }
    }


    /*
        Name of the function is loop.
        It runs a loop till 10
    */
    function loop() public {
        // for loop
        for (uint i = 0; i < 10; i++) {
            if (i == 3) {
                // Skip to next iteration with continue
                continue;
            }
            if (i == 5) {
                // Exit loop with break
                break;
            }
        }
    }
}
```